

Operators and Logic

Where We Left Off

A **variable** is one name for one value

A **struct** is a new type $\langle T \rangle$ built from members

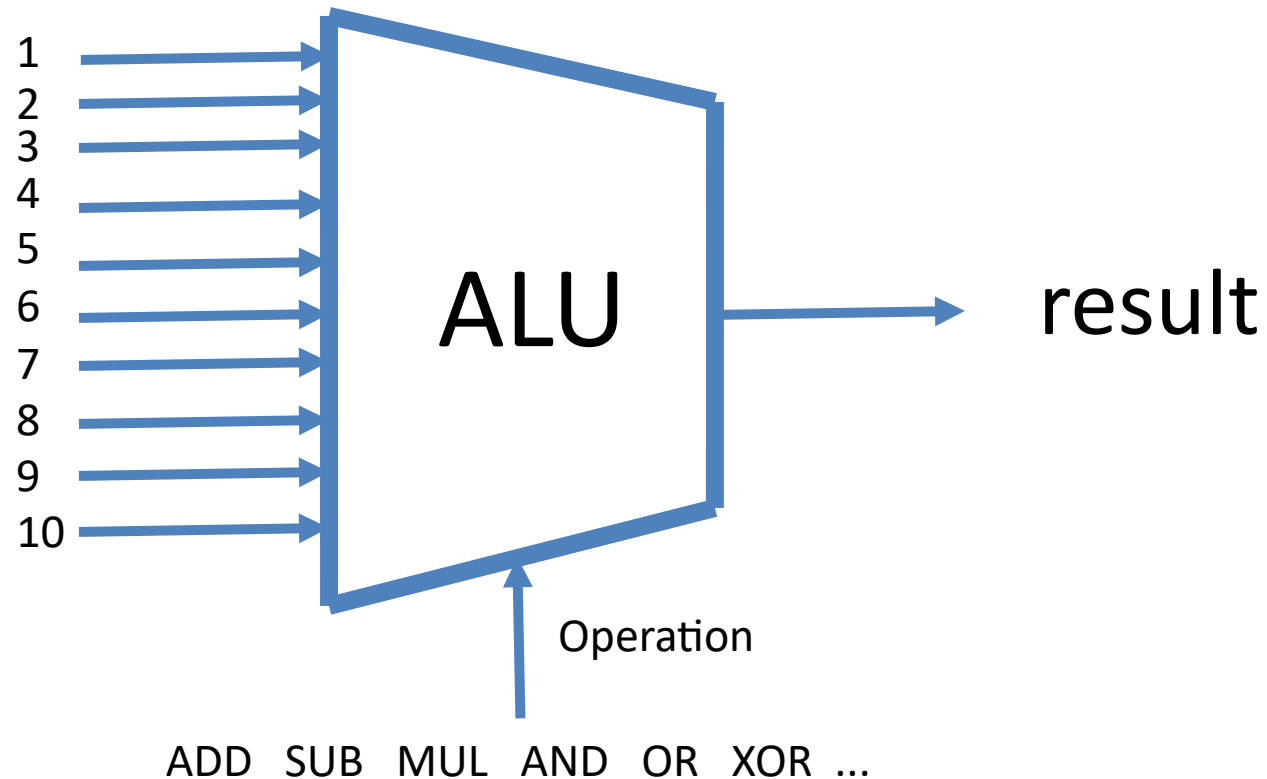
An **array** is N instances of one type

Now: what can we **do** with them?

The ALU Does the Work

*The ALU example did an **ADD***

*The **Operation** input selects what it does*



What an Operator Is

*An **operator** takes the contents of its **operands** and produces a new value*

(result) $\Leftarrow$ (a) + (b) Computer Architects view

Binary operators take two operands: $a + b$

Unary operators take one: $-a$ $!a$

Arithmetic Operators

+	<i>addition</i>
-	<i>subtraction</i>
*	<i>multiplication</i>
/	<i>division</i>
%	<i>remainder (modulo)</i>

*The **same symbols** in C, C++, and Rust*

Integer Division Truncates

$$7 / 2 = 3$$

$$-7 / 2 = -3$$

$$7.0 / 2.0 = 3.5$$

Integer division throws away the fraction

It drops the remainder

*It rounds **toward zero** in all three languages*

The Remainder Operator

$$7 \% 3 = 1$$

$$-7 \% 3 = -1 \quad \text{sign follows the left operand}$$

*C, C++: % is for **integers only***

Use `fmod(a, b)` for floating point

Rust: % also works on `f32` and `f64`

Division by Zero

Integers:

C, C++: **undefined behavior**

Rust: *the program **panics** and stops*

Floating point, all three languages:

1.0 / 0.0 = inf

0.0 / 0.0 = NaN (not a number)

Increment and Decrement

C, C++:

x++ ; add one

x-- ; subtract one

*Rust has **no ++ or -***

You will need to use the compound operators

x += 1 ;

x -= 1 ;

Prefix and Postfix

$x = 5; y = x++;$ y is 5, x is 6

$x = 5; y = ++x;$ y is 6, x is 6

*Postfix: use the value, **then** add one*

*Prefix: add one, **then** use the value*

Never use $x++$ twice in one expression

Compound Assignment

$x += 5;$	$(x) \leq (x) + 5$
$x -= 5;$	$(x) \leq (x) - 5$
$x *= 5;$	$(x) \leq (x) * 5$
$x /= 5;$	$(x) \leq (x) / 5$
$x \% = 5;$	$(x) \leq (x) \% 5$

Same in C, C++, and Rust

Overflow in C and C++

uint8_t x = 255;

x = x + 1; *x is now 0*

Unsigned *overflow wraps around - defined*

Signed *overflow is undefined behavior*

int8_t y = 127; y + 1 -> ???

Overflow in Rust

```
let x : u8 = 255 ;  
let y = x + 1 ;
```

Debug build:

panics - overflow is caught

Release build:

wraps to 0

Say what you mean:

returns

x.wrapping_add(1)

0

x.checked_add(1)

None

x.saturating_add(1)

255

Mixing Types in C and C++

uint8_t a = 200, b = 100 ;

int d = a + b ; 300

uint8_t c = a + b ; 44

*Small types are **promoted** to int first*

The result is cut down when stored

Mixing Types in Rust

let a : u8 = 200 ;

let d : i32 = a + 1000 ;

error

*Rust **never** converts types for you*

*You convert explicitly with **as**:*

let d : i32 = a as i32 + 1000 ;

Floating Point is Not Exact

$0.1 + 0.2 == 0.3$ **false**

Most decimals cannot be stored exactly in binary

Compare with a tolerance instead:

$fabs(a - b) < 1e-6$

C, C++

$(a - b).abs() < 1e-6$

Rust

Comparison Operators

`==` *equal*

`!=` *not equal*

`<` *less than*

`>` *greater than*

`<=` *less or equal*

`>=` *greater or equal*

The result is true or false

C: an int, 1 or 0

C++, Rust: a bool

= is not ==

*if (x = 5) assigns 5, then tests it
always true in C and C++*

if (x == 5) tests whether x is 5

*Rust refuses to compile the first one
an assignment is not a bool*

Chained Comparisons

Mathematics: $0 < x < 10$

C, C++: *compiles, and is **wrong***
($0 < x$) is 0 or 1 - always < 10

Rust: *compile error*

Write: $0 < x \ \&\& \ x < 10$

or better yet

$(0 < x) \ \&\& \ (x < 10)$

Operators on Struct Members

A member is an ordinary variable

Every operator works on it:

*my_sample.psi * 2.0*

my_sample.adc_counts + 1

my_sample.timestamp_ms - start

Same in C, C++, and Rust

Assigning a Whole Struct

C, C++:

```
sample_t sample2 ;  
sample2 = my_sample ;           copies every byte
```

Rust:

```
let sample2 = my_sample ;       moves it  
my_sample can no longer be used
```

Add #[derive(Clone, Copy)] to copy instead

Comparing Structs in C

if (my_sample == sample2)

compile error

Compare member by member:

my_sample.adc_counts == sample2.adc_counts

&& my_sample.psi == sample2.psi

&& my_sample.timestamp_ms == sample2.timestamp_ms

*Do not use memcmp - it compares the **padding** too*

Comparing Structs in C++

Ask the compiler to write == for you (C++20):

```
struct sample {  
    uint32_t    adc_counts ;  
    float      psi ;  
    uint64_t    timestamp_ms ;  
    bool operator == (const sample&) const = default ;  
};
```

Overloads == operator

Now my_sample == sample2 compares every member

// What the compiler effectively generates:

```
bool operator == (const sample& other) const {  
    return (adc_counts == other.adc_counts) &&  
        (psi == other.psi) &&  
        (timestamp_ms == other.timestamp_ms) ;  
}
```

Comparing Structs in Rust

```
#[derive(PartialEq)]  
struct Sample {  
    adc_counts : u32,  
    psi : f32,  
    timestamp_ms : u64,  
}
```

Now my_sample == sample2 compares every member

Arithmetic on Whole Structs

my_sample + sample2 ??

No language can guess what + means for your type

C: not possible - write a function

C++: define operator +

Rust: implement the Add trait

*This is **operator overloading** - later lecture possibly
(not a great idea IMHO)*

Operators on Array Elements

An element is an ordinary variable

Every operator works on it:

counts[i] + 1

*psi[i] * scale*

history[7].psi - history[6].psi

Same in C, C++, and Rust

Whole Arrays in C

uint32_t a[4], b[4];

a = b ;

compile error

if (a == b)

*compares **addresses**, not contents*

Copy with:

memcpy(a, b, sizeof(a)) ;

Compare with:

a loop, or memcmp

Whole Arrays in C++

#include <array>

std::array<uint32_t, 4> a, b ;

a = b ; *copies every element*

if (a == b) *compares every element*

Another reason to prefer std::array

Whole Arrays in Rust

let a : [u32 ; 4] = [1, 2, 3, 4] ;

let mut b : [u32 ; 4] = [0 ; 4] ;

b = a ; *copies every element*

if a == b { ... } *compares every element*

Works because the length is part of the type

Element-wise Math Needs a Loop

C, C++:

```
for ( int i = 0 ; i < 4 ; i++ ) {  
    sum[i] = a[i] + b[i] ;  
}
```

Rust:

```
for i in 0..4 {  
    sum[i] = a[i] + b[i] ;  
}
```

Two Kinds of Logic

Clause level

combines conditions that are true or false
if (psi < 10.0 || psi > 80.0)

Bit level

*applies logic to **every bit** of an integer*
status & 0x80

What Counts as True

- C:* *0 is false, **anything else** is true*
 if (count) means if (count != 0)
- C++:* *bool, but numbers convert silently*
- Rust:* *only a **bool** is a condition*
 if count { } **compile error**
 if count != 0 { } *correct*

Logical Operators

<code>&&</code>	<i>AND</i>	<i>true if both are true</i>
<code>//</code>	<i>OR</i>	<i>true if either is true</i>
<code>!</code>	<i>NOT</i>	<i>flips true and false</i>

*The **same symbols** in C, C++, and Rust*

```
if ( psi < 10.0 // psi > 80.0 ) alarm() ;
```

Short-Circuit Evaluation

$a \ \&\& \ b$ if a is false, b is **never evaluated**

$a \ || \ b$ if a is true, b is **never evaluated**

This makes guards safe:

if (count $\neq$ 0 && total / count > 10)

Division by zero cannot happen

Boolean Logic

*George Boole, 1854: logic as **algebra***

Two values: 0 and 1 (false and true)

*Every operation is defined completely by a
truth table: one row for every input combination*

Two inputs -> 4 rows

NOT

$$Y = A'$$

Flips the input

Clause level:

!a

all three

Bit level:

~a

C, C++

!a

Rust

A	Y
0	1
1	0

AND

$$Y = A \cdot B$$

*1 only when **both** inputs are 1*

Clause level:

a && b

all three

Bit level:

a & b

all three

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

$$Y = A + B$$

*1 when **either** input is 1
(or both)*

Clause level:

$a \vee b$

all three

Bit level:

$a \mid b$

all three

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NAND

$$Y = (A \cdot B)'$$

*NOT of AND: 0 only when
both inputs are 1*

Clause level:

!(a && b)

all three

Bit level:

~(a & b)

C, C++

!(a & b)

Rust

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR

$$Y = (A + B)'$$

*NOT of OR: 1 only when
both inputs are 0*

Clause level:

$!(a \parallel b)$

all three

Bit level:

$\sim(a \mid b)$

C, C++

$!(a \mid b)$

Rust

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XOR

$$Y = A \oplus B$$

*1 when the inputs **differ***

Clause level:

$a \neq b$

$a \wedge b$

Bit level:

$a \wedge b$

C, C++

Rust (bool)

all three

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

$$Y = (A \oplus B)'$$

*1 when the inputs are **equal***

Clause level:

$a == b$

all three

Bit level:

$\sim(a \wedge b)$

C, C++

$!(a \wedge b)$

Rust

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

All Seven Together

A	B	NOT A	AND	OR	NAND	NOR	XOR	XNOR
0	0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	1	0	0	0	1

NOT uses only A. The shaded columns are outputs.

NAND Can Build Everything

$$\text{NOT } A = A \text{ NAND } A$$

$$A \text{ AND } B = \text{NOT } (A \text{ NAND } B)$$

$$A \text{ OR } B = (\text{NOT } A) \text{ NAND } (\text{NOT } B)$$

*NAND is **universal***

A whole computer can be built from one kind of gate

De Morgan's Laws

$NOT (A AND B) = (NOT A) OR (NOT B)$

$NOT (A OR B) = (NOT A) AND (NOT B)$

In code:

$!(a \&\& b) == !a || !b$

$!(a || b) == !a \&\& !b$

Flip every operator and negate every term

Bitwise Operators

<code>&</code>	<i>AND</i>	<i>all three</i>
<code> </code>	<i>OR</i>	<i>all three</i>
<code>^</code>	<i>XOR</i>	<i>all three</i>
<code>~</code>	<i>NOT</i>	<i>C, C++</i>
<code>!</code>	<i>NOT</i>	<i>Rust</i>
<code><<</code>	<i>shift left</i>	<i>all three</i>
<code>>></code>	<i>shift right</i>	<i>all three</i>

*Each bit column is computed **independently***

Bitwise OR

Each column is one row of the OR truth table

*Result bit is 1 where **either** bit is 1*

	7	6	5	4	3	2	1	0	
<i>a</i>	1	1	0	0	1	0	1	0	$0xCA = 202$
<i>b</i>	1	1	1	1	0	0	0	0	$0xF0 = 240$
a b	1	1	1	1	1	0	1	0	$0xFA = 250$

Bitwise XOR

Each column is one row of the XOR truth table
*Result bit is 1 where the bits **differ***

	7	6	5	4	3	2	1	0	
<i>a</i>	1	1	0	0	1	0	1	0	$0xCA = 202$
<i>b</i>	1	1	1	1	0	0	0	0	$0xF0 = 240$
a ^ b	0	0	1	1	1	0	1	0	$0x3A = 58$

Bitwise NOT

Every bit flips

$\sim a$ in C and C++, $!a$ in Rust

	7	6	5	4	3	2	1	0	
a	1	1	0	0	1	0	1	0	$0xCA = 202$
$\sim a$	0	0	1	1	0	1	0	1	$0x35 = 53$

& is not &&

C: $1 \& 2 = 0$ *bitwise: 01 & 10 = 00*
 $1 \&\& 2 = 1$ *both are non-zero*

Rust: $1 \& 2 = 0$
 $1 \&\& 2$ **compile error** - *not bool*

if (a & b) is a different test than if (a && b)

Shift Left

Every bit moves one place left; a 0 enters on the right
 $x \ll n$ multiplies by 2 to the n

	7	6	5	4	3	2	1	0	
a	1	1	0	0	1	0	1	0	202
$a \ll 1$	1	0	0	1	0	1	0	0	148 (bit 7 fell off)

Shift Right

Every bit moves one place right; bit 0 falls off
 $x \gg n$ divides by 2 to the n

	7	6	5	4	3	2	1	0	
a	1	1	0	0	1	0	1	0	202
$a \gg 1$	0	1	1	0	0	1	0	1	101

Shifting Signed Values

Unsigned: a **0** enters on the left (logical)

Signed: the **sign bit** is copied (arithmetic)
 $-8 \gg 1 = -4$

Rust, C++20: guaranteed arithmetic
C: implementation defined

Use unsigned types for bit work

Shifting Too Far

`uint32_t x = 1 ;`

`x << 32` `??`

C, C++: **undefined behavior**

Rust: *panics in a debug build*

Shift by less than the width of the type

Use `1u << n` in C - never `1 << 31`

Bitwise Compound Assignment

$x \&= m ; \quad (x) \leq (x) \& m$

$x /= m ; \quad (x) \leq (x) / m$

$x \wedge= m ; \quad (x) \leq (x) \wedge m$

$x \ll= n ; \quad (x) \leq (x) \ll n$

$x \gg= n ; \quad (x) \leq (x) \gg n$

Same in C, C++, and Rust

Masks

A **mask** is a constant that selects bits

C: `#define ENABLE_BIT ( 1u << 7 )`

C++: `constexpr uint8_t ENABLE_BIT = 1u << 7 ;`

Rust: `const ENABLE_BIT : u8 = 1 << 7 ;`

*Constants are **SCREAMING_SNAKE_CASE**
in all three languages*

Set a Bit

$x |= (1 \ll 3);$ *OR forces bit 3 to 1*
Every other bit is unchanged

	7	6	5	4	3	2	1	0	
x	0	1	0	0	0	0	0	1	
$1 \ll 3$	0	0	0	0	1	0	0	0	<i>mask</i>
$x \text{mask}$	0	1	0	0	1	0	0	1	<i>bit 3 set</i>

Clear a Bit

$x \&= \sim(1 \ll 6);$
 $x \&= !(1 \ll 6);$

C, C++
Rust

	7	6	5	4	3	2	1	0	
x	0	1	0	0	1	0	0	1	
$\sim(1 \ll 6)$	1	0	1	1	1	1	1	1	<i>inverted mask</i>
$x \& \text{mask}$	0	0	0	0	1	0	0	1	<i>bit 6 cleared</i>

Toggle a Bit

$x \wedge= (1 \ll 0);$

XOR flips bit 0

Run it twice and you are back where you started

	7	6	5	4	3	2	1	0	
x	0	0	0	0	1	0	0	1	
$1 \ll 0$	0	0	0	0	0	0	0	1	<i>mask</i>
$x \wedge \text{mask}$	0	0	0	0	1	0	0	0	<i>bit 0 flipped</i>

Test a Bit

Shift it down, then keep only bit 0:

$(x \gg 3) \& 1$

1 or 0

Or mask it in place:

$(x \& (1u \ll 3)) \neq 0$

true or false

In Rust the second form gives a bool

The Precedence Trap

if (x & MASK == 0)

C, C++ read it as:

x & (MASK == 0)

always 0 - the test never passes

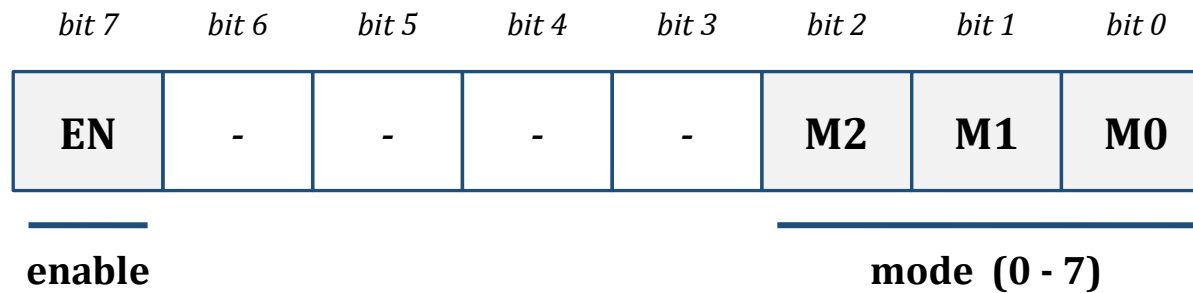
Rust reads it as:

(x & MASK) == 0

Always parenthesize bit operations

Fields Packed in a Register

One byte, two fields:



Reading and Writing Fields

mode = reg & 0x07 ;

low three bits

enable = (reg >> 7) & 1 ;

top bit

Change the mode, keep everything else:

reg = (reg & ~0x07) | (new_mode & 0x07) ;

Rust: the same, with ! in place of ~

Bit Operators on Structs

Members are integers - every bit operator applies:

my_sample.adc_counts & 0x0FFF

keep only the 12 bits a 12-bit ADC produces

my_sample.adc_counts >> 4

drop the 4 noisiest bits

Bit Operators on Arrays

A simple checksum: XOR every byte together

C, C++:

```
uint8_t check = 0 ;  
for ( int i = 0 ; i < N ; i++ ) check ^= data[i] ;
```

Rust:

```
let mut check : u8 = 0 ;  
for i in 0..N { check ^= data[i] ; }
```

Precedence in C and C++

Highest

! ~ ++ --

unary

** / %*

+ -

<< >>

< <= > >=

== !=

& then ^ then |

&& then ||

= += -= &= /= ...

Lowest

Precedence in Rust

Highest

! -

as

** / %*

+ -

<< >>

& then ^ then |

== != < <= > >=

&& then //

= += -= &= /= ...

Lowest

unary

type conversion

cannot chain

What We Can Now Do

Arithmetic *on variables, members, elements*

Comparison *values, and whole structs and arrays*

Clause logic *&& || ! on conditions*

Bit logic *& | ^ ~ << >> on every bit*

Seven truth tables describe all of it

Let's practice

*Exercise: Using **opspractice***